Week 8

Database Relationship Analysis using Joins

Scenario: The e-commerce company stores customer, product, order, and payment information in separate tables. The management team wants to combine these related tables to understand customer purchases, order details, and payment status.

1.1 What is a JOIN?

A JOIN in SQL is used to combine data from two or more tables using a related column between them. For example, we can use a JOIN to display customer names along with their order details from separate tables.

1.2 Types of SQL JOINS (FROM TASKS GIVEN)

1. INNER JOIN: Returns only the rows that have matching values in both tables.
2. LEFT JOIN: Returns all rows from the left table and matching rows from the right table; unmatched rows contain NULL values.
3. RIGHT JOIN: Returns all rows from the right table and matching rows from the left table; unmatched rows contain NULL values.

2. Database Tables and Attributes

The e-commerce database consists of multiple tables that store information about customers, orders, products, and payments. Each table represents a specific entity, and its attributes describe the information maintained about that entity.

2.1 Customer Table

Purpose: Stores customer information.

Attributes:

- Customer_ID – Unique identifier for each customer.
- Customer_Name – Name of the customer.
- Email – Customer's email address.
- City – City where the customer is located.

-

customer_id	customer_name	email	city
101	shiva	shiv@gmail.com	chennai
102	bhiba	bhi@gmail.com	chennai
103	suresh	shuri@gmail.com	chrompet
104	barsha	barsh@gmail.com	bengal
105	janhavi	jan@gmail.com	bengal
NULL	NULL	NULL	NULL

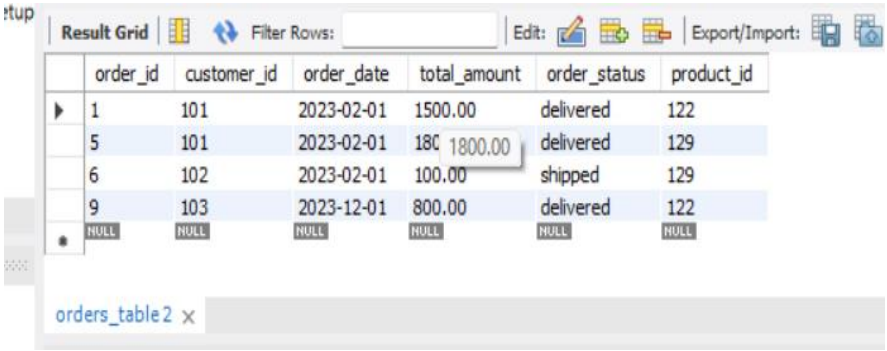
-

2.2 Orders Table

Purpose: Stores information about customer orders.

Attributes:

- Order_ID – Unique identifier for each order.
- Customer_ID – Identifies the customer who placed the order.
- Order_Date – Date on which the order was placed.
- Total_Amount – Total amount associated with the order.
- Order_Status – Current status of the order.
- Product_id

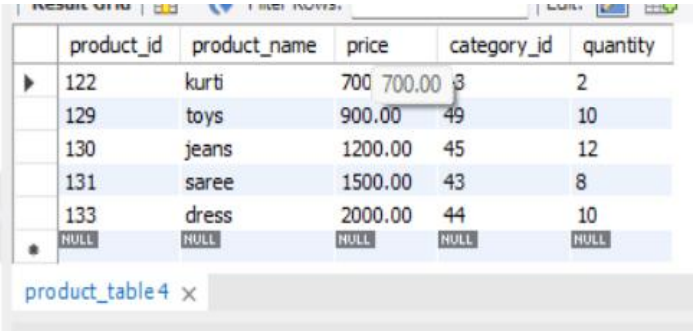


2.3 Product Table

Purpose: Stores information about the products available in the e-commerce system.

Attributes:

- Product_ID – Unique identifier for each product.
- Product_Name – Name of the product.
- Price – Price of the product.
- Category_ID – Identifier associated with the product's category.
- Quantity



product_id	product_name	price	category_id	quantity
122	kurti	700 700.00	3	2
129	toys	900.00	49	10
130	jeans	1200.00	45	12
131	saree	1500.00	43	8
133	dress	2000.00	44	10
NULL	NULL	NULL	NULL	NULL

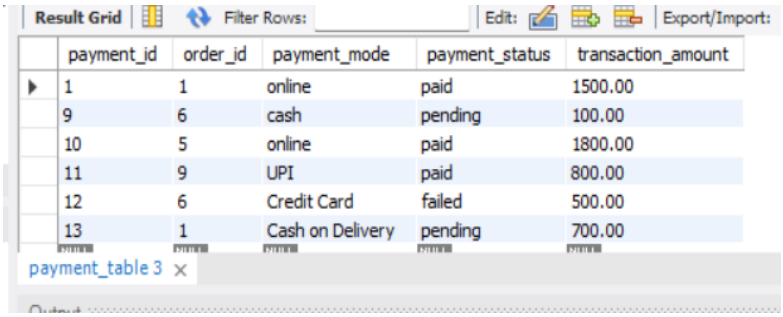
product_table 4

2.4 Payment Table

Purpose: Stores payment information associated with orders.

Attributes:

- Payment_ID – Unique identifier for each payment.
- Order_ID – Identifies the order associated with the payment.
- Payment_Mode – Mode of payment used.
- Payment_Status – Status of the payment.
- Transaction_amount



Result Grid | Filter Rows: | Edit: | Export/Import:

payment_id	order_id	payment_mode	payment_status	transaction_amount
1	1	online	paid	1500.00
9	6	cash	pending	100.00
10	5	online	paid	1800.00
11	9	UPI	paid	800.00
12	6	Credit Card	failed	500.00
13	1	Cash on Delivery	pending	700.00

payment_table 3

2.5 Relationships Between Tables

The tables are connected through related attributes:

- **Customer and Orders:** Connected using Customer_ID.
- **Orders and Payment:** Connected using Order_ID.
- **Product and Category:** The Category_ID in the Product table refers to a product category.

TASKS

INNER JOIN

1.Display customer details along with their orders.

```
23
24    #inner join
25    #Display customer details along with their orders.
26
27 •  select c.customer_id,c.customer_name,c.email,c.city,o.order_id,o.customer_id,o.order_date,o.total_amount,o.order_status
28    from customer_table c inner join orders_table o
29    on c.customer_id = o.customer_id;
30
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	customer_name	email	city	order_id	customer_id	order_date	total_amount	order_status
101	shiva	shiv@gmail.com	chennai	1	101	2023-02-01	1500.00	delivered
101	shiva	shiv@gmail.com	chennai	5	101	2023-02-01	1800.00	delivered
102	bhiba	bhi@gmail.com	chennai	6	102	2023-02-01	100.00	shipped
103	suresh	shuri@gmail.com	chrompet	9	103	2023-12-01	800.00	delivered

2. Display order details with payment information.

```
31    #Display order details with payment information.
32 •  select o.order_id,o.customer_id,o.order_date,o.total_amount,o.order_status,p.payment_id,p.order_id,p.payment_mode,p.payment_status
33    from orders_table o inner join payment_table p
34    on o.order_id=p.order_id;
35
36    #Retrieve products purchased by customers.
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

order_id	customer_id	order_date	total_amount	order_status	payment_id	order_id	payment_mode	payment_status
1	101	2023-02-01	1500.00	delivered	1	1	online	paid
6	102	2023-02-01	100.00	shipped	9	6	cash	pending
5	101	2023-02-01	1800.00	delivered	10	5	online	paid
9	103	2023-12-01	800.00	delivered	11	9	UPI	paid
6	102	2023-02-01	100.00	shipped	12	6	Credit Card	failed
1	101	2023-02-01	1500.00	delivered	13	1	Cash on Delivery	pending

Result 6 | Output

3.Retrieve products purchased by customers

```
36    #Retrieve products purchased by customers.
37
38 •  select c.customer_id,c.customer_name,c.email,c.city,o.order_id,o.customer_id,o.order_date,o.total_amount,o.order_status,p.payment_id,p.order_id,p.payment_mode,p.payment_status
39    from customer_table c inner join orders_table o
40    on c.customer_id=o.customer_id
41    inner join payment_table p
42    on o.order_id=p.order_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	customer_name	email	city	order_id	customer_id	order_date	total_amount	order_status	payment_id	order_id	payment_mode	payment_status
101	shiva	shiv@gmail.com	chennai	1	101	2023-02-01	1500.00	delivered	1	1	online	paid
102	bhiba	bhi@gmail.com	chennai	6	102	2023-02-01	100.00	shipped	9	6	cash	pending
101	shiva	shiv@gmail.com	chennai	5	101	2023-02-01	1800.00	delivered	10	5	online	paid
103	suresh	shuri@gmail.com	chrompet	9	103	2023-12-01	800.00	delivered	11	9	UPI	paid
102	bhiba	bhi@gmail.com	chennai	6	102	2023-02-01	100.00	shipped	12	6	Credit Card	failed
101	shiva	shiv@gmail.com	chennai	1	101	2023-02-01	1500.00	delivered	13	1	Cash on Delivery	pending

Result 7 | Read Only

Implement LEFT JOIN

1.Display all customers including customers who have not placed orders.

```
56    #Display all customers including customers who have not placed orders.
57 •  select c.* from customer_table c left join orders_table o on
58    c.customer_id=o.customer_id;
59
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	customer_name	email	city
101	shiva	shiv@gmail.com	chennai
101	shiva	shiv@gmail.com	chennai
102	bhiba	bhi@gmail.com	chennai
103	suresh	shuri@gmail.com	chrompet
104	barsha	barsh@gmail.com	bengal
105	janhavi	jan@gmail.com	bengal

Result 8 | Output

2. Find customers without any purchases.

```
60    #Find customers without any purchases.
61 •  select c.* from customer_table  c left join orders_table o on c.customer_id=o.customer_id where O.order_id IS NULL;
62
63
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	customer_name	email	city
104	barsha	barsh@gmail.com	bengal
105	janhavi	jan@gmail.com	bengal

3.Display all products including products with no sales.

```
69    #(here we can simply give product details but we used join bcz sepecial mention no sales)
70 •  select p.* from product_table p left join orders_table o
71    on  p.product_id=o.product_id;
72
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_id	product_name	price	category_id	quantity
122	kurti	700.00	43	2
122	kurti	700.00	43	2
129	toys	900.00	49	10
129	toys	900.00	49	10
130	jeans	1200.00	45	12
131	saree	1500.00	43	8

Result 10 | Output | Action Output

Implement RIGHT JOIN

1.Display all orders with customer information

```
#Display all orders with customer information.
select o.*,c.* from orders_table o right join customer_table c
on o.customer_id=c.customer_id;
```

order_id	customer_id	order_date	total_amount	order_status	product_id	customer_id	customer_name	email	city
5	101	2023-02-01	1800.00	delivered	129	101	shiva	shiv@gmail.com	chennai
1	101	2023-02-01	1500.00	delivered	122	101	shiva	shiv@gmail.com	chennai
6	102	2023-02-01	100.00	shipped	129	102	bhiba	bhi@gmail.com	chennai
9	103	2023-12-01	800.00	delivered	122	103	suresh	shuri@gmail.com	chrompet
NULL	NULL	NULL	NULL	NULL	NULL	104	barsha	barsh@gmail.com	bengal
NULL	NULL	NULL	NULL	NULL	NULL	105	janhavi	jan@gmail.com	bengal

2. Find orders where customer details are missing.

```
#Find orders where customer details are missing.
select o.*,c.* from customer_table c right join orders_table o
on c.customer_id = o.customer_id where c.customer_id is null;

# Display all payment records with order details.
```

order_id	customer_id	order_date	total_amount	order_status	product_id	customer_id	customer_name	email	city

4.Display all payment records with order details.

```
# Display all payment records with order details.

select p.*, o.* from payment_table p  inner join orders_table o
on p.order_id=o.order_id;
```

payment_id	order_id	payment_mode	payment_status	transaction_amount	order_id	customer_id	order_date	total_amount	order_status	product_id
1	1	online	paid	1500.00	1	101	2023-02-01	1500.00	delivered	122
9	6	cash	pending	100.00	6	102	2023-02-01	100.00	shipped	129
10	5	online	paid	1800.00	5	101	2023-02-01	1800.00	delivered	129
11	9	UPI	paid	800.00	9	103	2023-12-01	800.00	delivered	122
12	6	Credit Card	failed	500.00	6	102	2023-02-01	100.00	shipped	129
13	1	Cash on Delivery	pending	700.00	1	101	2023-02-01	1500.00	delivered	122

5.Retrieve Complete Order Details

EXPECTED:

Expected Output:

Customer	Product	Quantity	Amount	Payment Status
Arun	Laptop	1	55000	Success
Priya	Mobile	2	30000	Success

OUTPUT:

```
85
86    #report
87
88    select c.customer_name,p.product_name,p.quantity,o.total_amount,py.payment_status
89    from customer_table c inner join orders_table o
90    on c.customer_id=o.customer_id inner join payment_table py
91    on py.order_id=o.order_id inner join product_table p on p.product_id=o.product_id;
92
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_name	product_name	quantity	total_amount	payment_status
shiva	kurti	2	1500.00	paid
bhiba	toys	10	100.00	pending
shiva	toys	10	1800.00	paid
suresh	kurti	2	800.00	paid
bhiba	toys	10	100.00	failed
shiva	kurti	2	1500.00	pending

Result 25

6. Display Customer Purchase History Tasks

Write queries to find:

1.All products purchased by a customer.

```
92
93    #All products purchased by a customer.
94    select p.*,c.* from customer_table c  inner join orders_table o
95    on c.customer_id=o.customer_id inner join product_table p
96    on p.product_id=o.product_id where c.customer_id=101;
97
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_id	product_name	price	category_id	quantity	customer_id	customer_name	email	city
122	kurti	700.00	43	2	101	shiva	shiv@gmail.com	chennai
129	toys	900.00	49	10	101	shiva	shiv@gmail.com	chennai

Result 32

2. Total amount spent by each customer.

```
98    #Total amount spent by each customer
99
100   select customer_id,sum(total_amount) as total_Amount
101   from orders_table
102   group by customer_id;
103
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	total_Amount
101	3300.00
102	100.00
103	800.00

Result 33

3.Number of orders placed by each customer.

```
104   #Number of orders placed by each customer.
105   select customer_id,count(*) as no_of_orders
106   from orders_table
107   group by customer_id;
108
109   select count(*) from customer_table;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	no_of_orders
101	2
102	1
103	1

4. Latest purchase details of customers.

7. Generate Multi-Table Business Reports

Report 1:

●Customer Order Report Display

- **Customer name**
- **Order ID**
- **Order date**
- **Order status**

```
265   #Customer Order Report
266   select c.customer_name,o.order_id,o.order_date,o.order_status
267   from customer_table c inner join orders_table o
268   on c.customer_id=o.customer_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_name	order_id	order_date	order_status
shiva	1	2023-02-01	delivered
shiva	5	2023-02-01	delivered
bhiba	6	2023-02-01	shipped
suresh	9	2023-12-01	delivered

Result 36

Report 2: Sales Report Display:

- **Product name**
- **Quantity sold**
- **Total revenue**

```
select p.product_name,p.quantity,o.total_amount as total_revenue
from product_table p inner join orders_table o
on p.product_id=o.product_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_name	quantity	total_revenue
kurti	2	1500.00
toys	10	1800.00
toys	10	100.00
kurti	2	800.00

Result 40 | Output | Action Output

Report 3: Payment Analysis Report Display:

- **Payment mode**
- **Number of transactions**
- **Successful payments**

```
SELECT
    payment_mode,
    COUNT(*) AS no_of_transactions,
    SUM(CASE
        WHEN payment_status = 'paid' THEN 1
        ELSE 0
    END) AS successful_payments
FROM payment_table
GROUP BY payment_mode;
```

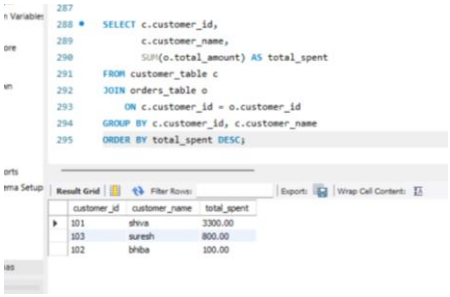
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

payment_mode	no_of_transactions	successful_payments
online	2	2
cash	1	0
UPI	1	1
Credit Card	1	0
Cash on Delivery	1	0

Result 42

Report 4: Customer Purchase Analysis Find:

- **Top purchasing customers**
- **Customers with maximum orders**
- **Customers with highest spending**



```
287
288 •  SELECT c.customer_id,
289           c.customer_name,
290           SUM(o.total_amount) AS total_spent
291    FROM customer_table c
292    JOIN orders_table o
293        ON c.customer_id = o.customer_id
294    GROUP BY c.customer_id, c.customer_name
295    ORDER BY total_spent DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	customer_name	total_spent
101	shiva	3300.00
103	suresh	800.00
102	bhiba	100.00